ProveIt Platform Documentation

1. Project Overview

ProveIt is a full-stack platform for collaborative technical interview practice. It combines problem solving, live sessions, scheduling, scoring, subscriptions, AI interviews, public blog content, and an admin dashboard.

The repository contains three main applications:

```
Provelt-Final/
  backend/   Express API, MongoDB models, integrations
  frontend/  React user-facing application
  admin/     Angular admin dashboard
```

2. Technology Stack

Frontend

- React 19
- Vite
- Tailwind CSS
- DaisyUI
- Clerk React
- Stream video/chat SDK
- Monaco Editor
- TanStack Query
- React Router
- Motion, Three.js, React Icons for home page visuals

Backend

- Node.js
- Express 5
- MongoDB with Mongoose
- Clerk Express middleware
- Stream Chat and Stream Video
- Judge0 code execution
- Resend email
- Inngest background events
- PDF parsing with pdf-parse and image fallback with mupdf

Admin

- Angular 17
- Angular Router
- Clerk JS
- Chart.js and ng2-charts
- Tailwind CSS
- DaisyUI

3. Application Architecture

```
Browser
  |
  | React frontend routes
  v
Frontend API layer
  |
  | HTTP REST + Clerk auth token
  v
Express backend
  |
  | Mongoose
  v
MongoDB

Express backend also integrates with:
  - Clerk for auth
  - Stream for chat/video
  - Judge0 for code execution
  - Resend for email
  - ITI/OpenAI-compatible gateway for AI features
  - Inngest for background jobs
```

4. Running Locally

4.1 Backend

```
cd backend
npm install
npm run dev
```

Backend entry point:

```
backend/src/server.js
```

4.2 Frontend

```
cd frontend
npm install
npm run dev
```

Frontend entry point:

```
frontend/src/main.jsx
frontend/src/App.jsx
```

4.3 Admin

```
cd admin
npm install
npm start
```

Admin entry point:

```
admin/src/main.ts
admin/src/app/app.routes.ts
```

4.4 Root Build

The root package.json includes:

```
npm run build
npm start
```

The root build installs backend/frontend dependencies and builds the frontend. The root start runs the backend.

5. Environment Variables

Backend environment variables are loaded in backend/src/lib/env.js.

Variable	Purpose

PORT	Backend port.
DB_URL	MongoDB connection string.
NODE_ENV	Runtime environment.
CLIENT_URL	Frontend URL used for CORS and session links.
CLERK_SECRET_KEY	Clerk backend authentication.
CLERK_PUBLISHABLE_KEY	Clerk public key.
STREAM_API_KEY	Stream API key.
STREAM_API_SECRET	Stream API secret.
JUDGE0_API_URL	Judge0 endpoint.
JUDGE0_API_KEY	Judge0/RapidAPI key.
JUDGE0_API_HOST	Judge0/RapidAPI host.
RESEND_API_KEY	Resend email key.
RESEND_FROM	Sender email identity.
OPENAI_MOCK_MODE	Enables mock AI behavior when true/1.
OPENAI_API_KEY	OpenAI key if used by provider.
ITI_API_KEY	ITI gateway key.
ITI_API_BASE_URL	ITI gateway base URL.
ITI_MODEL	Main ITI model.
ITI_FALLBACK_MODEL	Fallback ITI model.
ITI_VISION_MODEL	Vision model for scanned CVs.
INNGEST_EVENT_KEY	Inngest event key.
INNGEST_SIGNING_KEY	Inngest signing key.
ADMIN_EMAIL	Admin email configuration.

6. Backend API

6.1 Authentication

Most API routes use protectRoute, which depends on Clerk middle
a MongoDB user.

Admin routes use adminAuth.

6.2 API Route Groups

Route Group	File	Description
/api/users	backend/src/routes/userRoute.js	Leaderboard, current score summary, solved problems.
/api/subscription	backend/src/routes/subscriptionRoute.js	Plan status, simulated payment, cancellation, AI interview limit check.

/api/sessions	backend/src/routes/sessionRoute.js	Live session lifecycle.
/api/notifications	backend/src/routes/notificationRoute.js	Notification list, read, read-all, delete.
/api/invitations	backend/src/routes/invitationRoute.js	Search users, send invitation, respond, list invitations.
/api/code	backend/src/routes/codeRoute.js	Execute code through Judge0.
/api/chat	backend/src/routes/chatRoute.js	Stream chat token.
/api/availability	backend/src/routes/availabilityRoute.js	Weekly availability.
/api/ai-interview	backend/src/routes/aiInterviewRoute.js	CV upload, AI interview start/answer/results/history.
/api/blog	backend/src/routes/blogRoute.js	Public blog listing/detail.
/api/admin	backend/src/routes/adminRoute.js	Admin dashboard, users, sessions, AI interviews, blog posts.

7. Frontend Routes

File: frontend/src/App.jsx

Route	Access	Page
/	Public	Home
/pricing	Public	Pricing
/about	Public	About
/blog	Public	Blog listing
/blog/:slug	Public	Blog detail
/contact	Public	Contact
/dashboard	Authenticated	User dashboard
/problems	Authenticated	Problem list
/problem/:id	Authenticated	Problem workspace
/session/:id	Authenticated	Live session
/availability/settings	Authenticated	Availability settings
/schedule	Authenticated	Schedule session
/invitations	Authenticated	Invitations
/subscription	Authenticated	Subscription
/ai-interview	Authenticated	AI interview hub
/ai-interview/upload-cv	Authenticated	CV upload
/ai-interview/session/:sessionId	Authenticated	AI interview workspace
/ai-interview/results/:sessionId	Authenticated	Results
*	Public	404

8. Admin Routes

File: admin/src/app/app.routes.ts

Route	Description
/login	Admin login
/dashboard	Dashboard stats and charts
/users	User list and role management
/sessions	Session list
/ai-interviews	AI interview list
/blog-posts	Blog post CRUD

9. Core Feature Documentation

9.1 User Dashboard

The dashboard shows:

- Welcome/user information
- Score and rank
- Solved problems
- Recent sessions
- Active sessions
- Leaderboard preview

Relevant files:

- frontend/src/pages/DashboardPage.jsx
- frontend/src/components/StatsCards.jsx
- frontend/src/components/SolvedProblems.jsx
- frontend/src/components/RecentSessions.jsx
- frontend/src/components/ActiveSessions.jsx
- frontend/src/components/Leaderboard.jsx

9.2 Problems and Scoring

Practice problems live in:

`frontend/src/data/problems.js`

Execution endpoint:

`POST /api/code/execute`

Code execution flow:

1. User submits code from problem page/session workspace.
2. Frontend sends language, code, problem id/title, difficulty, and ex
3. Backend sends code to Judge0.
4. Backend compares stdout to expected output.
5. Backend creates CodeSubmission.
6. Backend recalculates User.score, User.level, and User.scoreStat
7. Frontend may show score toast when score changes.

Scoring:

- Easy solved: 10
- Medium solved: 20
- Hard solved: 35
- First try bonus: 5
- Daily bonus: 3
- Wrong penalty: -2

9.3 Sessions

Session creation no longer requires a predefined problem. A user chooses only difficulty:

- easy
- medium
- hard

Inside a session, users choose a problem from that difficulty. The selected problem is saved to the Session document as:

- selectedProblemId
- problem

Relevant files:

- backend/src/controllers/sessionController.js
- backend/src/models/Session.js
- frontend/src/pages/SessionPage.jsx
- frontend/src/components/CreateSessionModal.jsx

9.4 Scheduling and Invitations

Scheduling uses weekly availability.

Availability route:

```
GET /api/availability/me
PUT /api/availability/me
GET /api/availability/:userId
```

Invitation route:

```
GET /api/invitations/search-users
POST /api/invitations
GET /api/invitations/me
POST /api/invitations/:invitationId/respond
```

Acceptance flow:

1. Sender sends invitation for date, time, duration, difficulty.
2. Recipient receives in-app notification and email.
3. Recipient accepts.
4. Backend creates a full Session.
5. Backend creates Stream video call and chat channel.
6. Sender and recipient receive confirmation email.

9.5 Notifications

Notifications support:

- Fetch my notifications
- Mark one as read
- Mark all as read
- Delete one notification

Route:

```
GET /api/notifications
PATCH /api/notifications/:id/read
PATCH /api/notifications/read-all
DELETE /api/notifications/:id
```

9.6 AI Interview

AI interview workflow:

1. Upload PDF CV.
2. Backend parses PDF text.
3. If text cannot be extracted, backend tries image-based CV analysis.
4. Backend stores or updates CandidateProfile.
5. User starts interview with optional target role/job description.
6. Backend prepares personalized spoken questions.
7. User submits answers.
8. Backend evaluates each answer and may ask follow-up.
9. Backend generates final results.
10. Completed event is sent through Inngest for profile progress tracking.

Relevant files:

- backend/src/controllers/aiInterviewController.js
- backend/src/ai/aiInterviewService.js
- backend/src/lib/openai.js
- frontend/src/pages/AiInterviewHubPage.jsx
- frontend/src/pages/CvUploadPage.jsx
- frontend/src/pages/InterviewPage.jsx
- frontend/src/pages/ResultsPage.jsx

9.7 Blog

Public blog routes:

```
GET /api/blog
GET /api/blog/:slug
```

Admin blog routes:

```
GET /api/admin/blog-posts
POST /api/admin/blog-posts
PUT /api/admin/blog-posts/:id
DELETE /api/admin/blog-posts/:id
```

Blog posts support English and Arabic localized content.

9.8 Subscriptions

Plans:

- Free
- Pro
- Premium

Frontend plan definitions:

```
frontend/src/data/plans.js
```

Backend subscription route:

```
GET /api/subscription/me
POST /api/subscription/simulate-payment
POST /api/subscription/cancel
GET /api/subscription/check-limit
```

Payment is simulated. No real payment gateway is currently integrated.

10. Internationalization and Themes

Language state:

`frontend/src/context/LanguageContext.jsx`

Translations:

`frontend/src/lib/translations.js`

The app supports:

- English
- Arabic
- LTR and RTL document direction

Themes:

```
frontend/src/context/ThemeContext.jsx
frontend/src/lib/themes.js
frontend/src/components/ThemeSwitcher.jsx
```

DaisyUI semantic classes are used for theme compatibility where possible.

11. Deployment Notes

Backend production behavior:

- Serves static frontend from frontend/dist when NODE_ENV === "production".
- Falls back to frontend index.html for non-API routes.

Frontend public redirects:

- frontend/public/_redirects

Admin redirects:

- admin/src/_redirects
- admin/public/_redirects

CORS origins are configured in:

`backend/src/server.js`

12. Testing and Verification

Common local checks:

```
cd frontend
npm run lint
npm run build
```

```
cd admin
npm run build
```

```
cd backend
npm run start
```

Recommended manual flows:

1. Sign in and load dashboard.
2. Solve a problem and verify score update.
3. Create a session by difficulty.

4. Join a session and select a problem.
5. Set availability and schedule an invitation.
6. Accept invitation and join created session.
7. Upload CV and complete AI interview.
8. View blog listing and detail in English and Arabic.
9. Use admin dashboard to view stats and manage blog posts.

13. Known Constraints

- Practice problems are frontend static data rather than database records.
- Payment is simulated.
- AI interview quota enforcement is present but disabled in aiInterviewController.js.
- Email depends on Resend environment configuration.
- Stream video/chat requires valid Stream credentials.
- Judge0 availability depends on external service configuration.